

[1회차] Python 과제

발제자: 김아연

기한

- 과제 마감: 7월 13일 월요일 23시 59분까지
- 지각 제출: 7월 14일 화요일 23시 59분까지

Intro

안녕하세요!

이번 과제의 목적은 **자료구조·알고리즘을 직접 구현하고, 코드 구조를 이해하는** 것입니다.

첫 과제는 백준 문제 풀이로 구성되어 있으며, 총 **10문제**입니다.

난이도는 **실버부터 플래티넘까지** 섞여 있습니다.

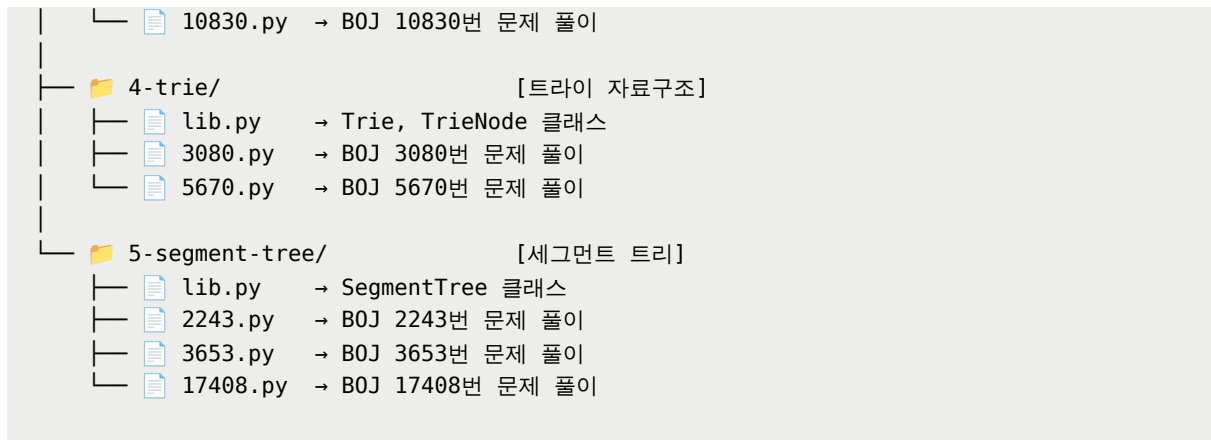
채점 기준을 참고하여 필요한 개수만 선택해서 풀어도 되고, 전부 풀고 싶은 분들은 자유롭게 도전해 주세요!

GPT 등의 LLM을 사용하지 않고, 최대한 혼자 힘으로 구현해보는 것을 권장합니다.

과제 명세1. 과제 파일 구성

아래와 같은 파일 구조를 기준으로 과제를 진행합니다.

```
1(1)-Python/
├── convert_for_submission.py
├── 1-graph-traversal/           [그래프 탐색]
│   ├── lib.py                 → Graph 클래스 (DFS, BFS 구현)
│   └── 1260.py                 → BOJ 1260번 문제 풀이
├── 2-stack-queue-deque/        [스택/큐/덱]
│   ├── lib.py                 → 원형 큐 관련 함수
│   ├── 2164.py                 → BOJ 2164번 문제 풀이
│   └── 11866.py                → BOJ 11866번 문제 풀이
├── 3-divide-and-conquer-multiplication/ [분할정복/행렬]
│   ├── lib.py                 → Matrix 클래스 (행렬 연산)
│   └── 1629.py                 → BOJ 1629번 문제 풀이
```



2. 문제 풀이 방식

1) 각 파일에서 `# 구현하세요!` 라고 작성되어 있는 부분을 채워주세요.

- 파일별 `TODO` 블록 안의 instruction을 참고해 주세요.
- `# 아무것도 수정하지 마세요!` 라고 작성된 블록은 수정하지 마세요. (오류 발생)
- 기본 제공되는 템플릿을 사용하지 않을 경우 제출로 인정되지 않습니다.
- 템플릿 코드 수정은 `mypy` 를 위한 type 수정까지만 허용됩니다.

2) lib.py 사용 규칙

같은 자료구조를 사용하는 문제들은 모두 같은 폴더의 `lib.py` 에 있는 동일한 구현체를 기반으로 풀어야 합니다.

- 범용성 높게 구현하여 코드의 재사용성을 최대화해 봅시다
- (예외) `1629.py` 는 같은 폴더의 `lib.py` 에 정의된 `Matrix` 클래스와 무관합니다.

3) Docstring 작성

구현한 메소드에는 **docstring**을 작성해 주세요. 세션 때 배운 내용을 복습하는 목적입니다.

구현 후 정답 확인 방법

1. mypy 테스트 실행

파일별로 `mypy` 테스트를 실행했을 때 `Success` 가 뜨는지 확인해 주세요.

2. 제출용 코드 생성

제출용 코드는 `convert_for_submission.py` 를 통해 생성할 수 있습니다.

1. `submission` 폴더를 생성합니다.

2. 아래 명령어를 실행합니다.

전체 문제 제출용 파일 생성:

```
python convert_for_submission.py
```

개별 문제 제출용 파일 생성:

```
python convert_for_submission.py {문제번호}
```

예시: python convert_for_submission.py1260

3. 정답 확인

정답 확인은 **CS 기초 과제**의 `script.sh` 실행 결과로 대체합니다.

문제 목록

난이도는 백준 기준입니다.

1. 그래프와 순회

▼ 1260번 - DFS와 BFS (실II)

문제

그래프를 DFS로 탐색한 결과와 BFS로 탐색한 결과를 출력하는 프로그램을 작성하세요.

단, 방문할 수 있는 정점이 여러 개인 경우에는 정점 번호가 작은 것을 먼저 방문하고, 더 이상 방문할 수 있는 점이 없는 경우 종료한다.

정점 번호는 1번부터 N번까지이다.

입력

첫째 줄에 정점의 개수 $N(1 \leq N \leq 1,000)$, 간선의 개수 $M(1 \leq M \leq 10,000)$, 탐색을 시작할 정점의 번호 V 가 주어진다.

다음 M개의 줄에는 간선이 연결하는 두 정점의 번호가 주어진다.

어떤 두 정점 사이에 여러 개의 간선이 있을 수 있다.

입력으로 주어지는 간선은 양방향이다.

출력

첫째 줄에 DFS를 수행한 결과를, 그 다음 줄에는 BFS를 수행한 결과를 출력한다.

V부터 방문된 점을 순서대로 출력하면 된다.

2. 스택, 큐, 덱

▼ 2164번 - 카드2 (실Ⅳ)

문제

N장의 카드가 있다. 각각의 카드는 차례로 1부터 N까지의 번호가 붙어 있으며, 1번 카드가 제일 위에, N번 카드가 제일 아래인 상태로 순서대로 카드가 놓여 있다.

이제 다음과 같은 동작을 카드가 한 장 남을 때까지 반복하게 된다.

우선, 제일 위에 있는 카드를 바닥에 버린다.

그 다음, 제일 위에 있는 카드를 제일 아래에 있는 카드 밑으로 옮긴다.

예를 들어 N=4인 경우를 생각해 보자.

카드는 제일 위에서부터 1 2 3 4의 순서로 놓여 있다.

1을 버리면 2 3 4가 남는다.

여기서 2를 제일 아래로 옮기면 3 4 2가 된다.

3을 버리면 4 2가 되고, 4를 밑으로 옮기면 2 4가 된다.

마지막으로 2를 버리고 나면, 남는 카드는 4가 된다.

N이 주어졌을 때, 제일 마지막에 남게 되는 카드를 구하는 프로그램을 작성하세요.

입력

첫째 줄에 정수 $N(1 \leq N \leq 500,000)$ 이 주어진다.

출력

첫째 줄에 남게 되는 카드의 번호를 출력한다.

▼ 11866번 - 요세푸스 문제 0 (실IV)

문제

요세푸스 문제는 다음과 같다.

1번부터 N번까지 N명의 사람이 원을 이루면서 앉아 있고, 양의 정수 $K(\leq N)$ 가 주어진다.

이제 순서대로 K번째 사람을 제거한다.

한 사람이 제거되면 남은 사람들로 이루어진 원을 따라 이 과정을 계속 나간다.

이 과정은 N명의 사람이 모두 제거될 때까지 계속된다.

원에서 사람들이 제거되는 순서를 (N, K)-요세푸스 순열이라고 한다.

예를 들어 (7, 3)-요세푸스 순열은 $\langle 3, 6, 2, 7, 5, 1, 4 \rangle$ 이다.

N과 K가 주어지면 (N, K)-요세푸스 순열을 구하는 프로그램을 작성하세요.

입력

첫째 줄에 N과 K가 빈 칸을 사이에 두고 순서대로 주어진다.

$(1 \leq K \leq N \leq 1,000)$

출력

예제와 같이 요세푸스 순열을 출력한다.

3. 분할 정복을 이용한 거듭제곱

▼ 1629번 - 곱셈 (실)

문제

자연수 A를 B번 곱한 수를 알고 싶다.

단 구하려는 수가 매우 커질 수 있으므로 이를 C로 나눈 나머지를 구하는 프로그램을 작성하세요.

입력

첫째 줄에 A, B, C가 빈 칸을 사이에 두고 순서대로 주어진다.

A, B, C는 모두 2,147,483,647 이하의 자연수이다.

출력

첫째 줄에 A를 B번 곱한 수를 C로 나눈 나머지를 출력한다.

▼ 10830번 - 행렬 제곱 (골IV)

문제

크기가 $N \times N$ 인 행렬 A가 주어진다.

이때, A의 B제곱을 구하는 프로그램을 작성하세요.

수가 매우 커질 수 있으니, A^B 의 각 원소를 1,000으로 나눈 나머지를 출력한다.

입력

첫째 줄에 행렬의 크기 N과 B가 주어진다.

$(2 \leq N \leq 5, 1 \leq B \leq 100,000,000,000)$

둘째 줄부터 N개의 줄에 행렬의 각 원소가 주어진다.

행렬의 각 원소는 1,000보다 작거나 같은 자연수 또는 0이다.

출력

첫째 줄부터 N개의 줄에 걸쳐 행렬 A를 B제곱한 결과를 출력한다.

4. 트라이

▼ 3080번 - 아름다운 이름 (풀III)

문제

상근 선생님은 학생들에게 번호를 붙여주려고 한다.

상근이는 미술 선생님이기 때문에, 이름의 순서도 아름다워야 한다고 생각한다.

따라서, 다음과 같은 규칙을 지켜서 번호를 정하려고 한다.

같은 알파벳 순서로 시작하는 두 이름의 사이에는 모두 그 순서로 시작하는 단어가 있어야 한다.

예를 들어, 두 이름이 MARTHA와 MARY는 모두 MAR로 시작한다.

따라서, 그 두 단어 사이에는 MARCO나 MARVIN과 같이 MAR로 시작하는 이름만이 존재할 수 있다.

MAY는 그 사이에 있을 수 없다.

상근이의 규칙을 지키면서 학생들 이름의 순서를 정하는 방법의 수를 구하는 프로그램을 작성하세요.

참고로 사전순으로 정렬하면, 항상 상근이의 규칙을 지킬 수 있다.

입력

첫째 줄에 이름의 수 N 이 주어진다. ($3 \leq N \leq 3000$)

다음 N 개 줄에는 이름이 한 줄에 하나씩 주어진다.

이름의 길이는 3000보다 작으며, 알파벳 대문자로만 이루어져 있다.

모든 이름은 중복되지 않는다.

출력

상근이의 규칙을 지키면서 순서를 정하는 방법의 수를 1,000,000,007로 나눈 나머지를 출력한다.

▼ 5670번 - 휴대폰 자판 (플IV)

문제

휴대폰에서 길이가 P인 영단어를 입력하려면 버튼을 P번 눌러야 한다.

그러나 시스템프로그래밍 연구실에 근무하는 승혁연구원은 사전을 사용해 이 입력을 더 빨리 할 수 있는 자판 모듈을 개발하였다.

이 모듈은 사전 내에서 가능한 다음 글자가 하나뿐이라면 그 글자를 버튼 입력 없이 자동으로 입력해 준다.

자세한 작동 과정은 다음과 같다.

1. 모듈이 단어의 첫 번째 글자를 추론하지는 않는다.
즉, 사전의 모든 단어가 같은 알파벳으로 시작하더라도 반드시 첫 글자는 사용자가 버튼을 눌러 입력해야 한다.
2. 길이가 1 이상인 문자열 $c_1c_2...c_n$ 이 지금까지 입력되었을 때,
사전 안의 모든 $c_1c_2...c_n$ 으로 시작하는 단어가 $c_1c_2...c_nc$ 로도 시작하는 글자 c 가 존재한다면
모듈은 사용자의 버튼 입력 없이 자동으로 c 를 입력해 준다.
그렇지 않다면 사용자의 입력을 기다린다.

예를 들면, 사전에 "hello", "hell", "heaven", "goodbye" 4개의 단어가 있고 사용자가 "h"를 입력하면 모듈은 자동으로 "e"를 입력해 준다.

사전상의 "h"로 시작하는 단어들은 모두 그 뒤에 "e"가 오기 때문이다.

그러나 단어들 중 "hel"로 시작하는 것도, "hea"로 시작하는 것도 있기 때문에 여기서 모듈은 사용자의 입력을 기다린다.

이어서 사용자가 "l"을 입력하면 모듈은 "l"을 자동으로 입력해 준다.

그러나 여기서 끝나는 단어인 "hell"과 그렇지 않은 단어인 "hello"가 공존하므로 모듈은 다시 입력을 기다린다.

사용자가 "hell"을 입력하기 원한다면 여기서 입력을 멈출 것이고,

"hello"를 입력하기 원한다면 직접 "o" 버튼을 눌러 입력해 줘야 한다.

따라서 "hello"를 입력하려면 사용자는 총 3번 버튼을 눌러야 하고, "hell", "heaven"은 2번이다.

"heaven"의 경우 "he"에서 "a"를 입력하면 바로 뒷부분이 모두 자동으로 입력되기 때문이다.

비슷하게, "goodbye"는 단 한 번만 버튼을 눌러도 입력이 완료될 것이다.

"g"를 입력하는 순간 뒤에 오는 글자가 항상 유일하므로 끝까지 자동으로 입력되기 때문이다.

이때 사전에 있는 단어들을 입력하기 위해 버튼을 눌러야 하는 횟수의 평균은 $(3 + 2 + 2 + 1) / 4 = 2.00$ 이다.

사전이 주어졌을 때, 이 모듈을 사용하면서 이와 같이 각 단어를 입력하기 위해 버튼을 눌러야 하는 횟수의 평균을 구하는 프로그램을 작성하세요.

입력

입력은 여러 개의 테스트 케이스로 이루어져 있다.

각 테스트 케이스의 첫째 줄에 사전에 속한 단어의 개수 N 이 주어지며($1 \leq N \leq 10^5$),

이어서 N 개의 줄에 1~80글자인 영어 소문자로만 이루어진 단어가 하나씩 주어진다.

이 단어들로 사전이 구성되어 있으며, 똑같은 단어는 두 번 주어지지 않는다.

각 테스트 케이스마다 입력으로 주어지는 단어의 길이 총합은 최대 10^6 이다.

출력

각 테스트 케이스마다 한 줄에 걸쳐 문제의 정답을 소수점 둘째 자리까지 반올림하여 출력한다.

5. 세그먼트 트리

▼ 2243번 - 사탕상자 (플V)

문제

수정이는 어린 동생을 달래기 위해서 사탕을 사용한다.

수정이는 평소에 여러 개의 사탕을 사서 사탕상자에 넣어두고, 동생이 말을 잘 들을 때면 그 안에서 사탕을 꺼내서 주곤 한다.

각각의 사탕은 그 맛의 좋고 나쁨이 1부터 1,000,000까지의 정수로 구분된다.

1이 가장 맛있는 사탕을 의미하며, 1,000,000은 가장 맛없는 사탕을 의미한다.

수정이는 동생이 말을 잘 들은 정도에 따라서, 사탕상자 안에 있는 사탕들 중 몇 번째로 맛있는 사탕을 꺼내주곤 한다.

예를 들어 말을 매우 잘 들었을 때에는 사탕상자에서 가장 맛있는 사탕을 꺼내주고, 말을 조금 잘 들었을 때에는 사탕상자에서 여섯 번째로 맛있는 사탕을 꺼내주는 식이다.

수정이가 보관하고 있는 사탕은 매우 많기 때문에 매번 사탕상자를 뒤져서 꺼낼 사탕을 골라내는 일은 매우 어렵다.

수정이를 도와주는 프로그램을 작성하세요.

입력

첫째 줄에 수정이가 사탕상자에 손을 댄 횟수 n ($1 \leq n \leq 100,000$)이 주어진다.

다음 n 개의 줄에는 두 정수 A , B , 혹은 세 정수 A , B , C 가 주어진다.

A 가 1인 경우는 사탕상자에서 사탕을 꺼내는 경우이다.

이때에는 한 정수만 주어지며, B 는 꺼낼 사탕의 순위를 의미한다.

이 경우 사탕상자에서 한 개의 사탕이 꺼내지게 된다.

A 가 2인 경우는 사탕을 넣는 경우이다.

이때에는 두 정수가 주어지는데, B 는 넣을 사탕의 맛을 나타내는 정수이고 C 는 그러한 사탕의 개수이다.

C 가 양수일 경우에는 사탕을 넣는 경우이고, 음수일 경우에는 빼는 경우이다.

맨 처음에는 빈 사탕상자에서 시작한다고 가정하며, 사탕의 총 개수는 2,000,000,000을 넘지 않는다.

또한 없는 사탕을 꺼내는 경우와 같은 잘못된 입력은 주어지지 않는다.

출력

A 가 1인 모든 입력에 대해서, 꺼낼 사탕의 맛의 번호를 출력한다.

예제 입력 1

```
6
2 1 2
2 3 3
1 2
1 2
2 1 -1
1 2
```

예제 출력 1

```
1
3
3
```

▼ 3653번 - 영화 수집 (플IV)

문제

상근이는 영화 DVD 수집가이다.

상근이는 그의 DVD 컬렉션을 쌓아 보관한다.

보고 싶은 영화가 있을 때는, DVD의 위치를 찾은 다음 쌓아놓은 컬렉션이 무너지지 않게 조심스럽게 DVD를 뺀다.

영화를 다 본 이후에는 가장 위에 놓는다.

상근이는 DVD가 매우 많기 때문에, 영화의 위치를 찾는 데 시간이 너무 오래 걸린다.

각 DVD의 위치는, 찾으려는 DVD의 위에 있는 영화의 개수만 알면 쉽게 구할 수 있다.

각 영화는 DVD 표지에 붙어있는 숫자로 쉽게 구별할 수 있다.

각 영화의 위치를 기록하는 프로그램을 작성하세요.

상근이가 영화를 한 편 볼 때마다 그 DVD의 위에 몇 개의 DVD가 있었는지를 구해야 한다.

입력

첫째 줄에 테스트 케이스의 개수가 주어진다.

테스트 케이스의 개수는 100개를 넘지 않는다.

각 테스트 케이스의 첫째 줄에는 상근이가 가지고 있는 영화의 수 n 과 보려고 하는 영화의 수 m 이 주어진다.

$(1 \leq n, m \leq 100,000)$

둘째 줄에는 보려고 하는 영화의 번호가 순서대로 주어진다.

영화의 번호는 1부터 n 까지이며, 가장 처음에 영화가 쌓여진 순서는 1부터 증가하는 순서이다.

가장 위에 있는 영화의 번호는 1이다.

출력

각 테스트 케이스에 대해서 한 줄에 m 개의 정수를 출력해야 한다.

i 번째 출력하는 수는 i 번째로 영화를 볼 때 그 영화의 위에 있었던 DVD의 개수이다.

상근이는 매번 영화를 볼 때마다 본 영화 DVD를 가장 위에 놓는다.

▼ 17408번 - 수열과 쿼리 24 (플IV)

문제

길이가 N 인 수열 $A_1, A_2, \dots, A_N$ 이 주어진다.

이때, 다음 쿼리를 수행하는 프로그램을 작성하시오.

```
1 i v:  $A_i$ 를  $v$ 로 바꾼다. ( $1 \leq i \leq N, 1 \leq v \leq 10^9$ )
2 l r:  $l \leq i < j \leq r$ 을 만족하는 모든  $A_i + A_j$  중에서 최댓값을 출력한다. ( $1 \leq l < r \leq N$ )
```

수열의 인덱스는 1부터 시작한다.

입력

첫째 줄에 수열의 크기 N 이 주어진다.

($2 \leq N \leq 100,000$)

둘째 줄에는 $A_1, A_2, \dots, A_N$ 이 주어진다.

($1 \leq A_i \leq 10^9$)

셋째 줄에는 쿼리의 개수 M 이 주어진다.

($2 \leq M \leq 100,000$)

넷째 줄부터 M 개의 줄에는 쿼리가 한 줄에 하나씩 주어진다.

출력

2번 쿼리에 대해서 정답을 한 줄에 하나씩 순서대로 출력한다.

제출 방법

과제 제출용 GitHub repo에 아래 파일 구조로 제출

```
YBIGTA_newbie_assignment/
├── 1(1)-Python/
│   ├── 1-graph-traversal/
│   │   ├── 1260.py
│   │   └── lib.py
│   └── 2-stack-queue-deque/
│       ├── 2164.py
│       ├── 11866.py
│       └── lib.py
```

```

├── 3-divide-and-conquer-multiplication/
│   ├── 1629.py
│   ├── 10830.py
│   └── lib.py
├── 4-trie/
│   ├── 3080.py
│   ├── 5670.py
│   └── lib.py
├── 5-segment-tree/
│   ├── 2243.py
│   ├── 3653.py
│   ├── 17408.py
│   └── lib.py
├── convert_for_submission.py
└── submission/
    ├── 1_1260.py
    ├── 2_2164.py
    ├── 2_11866.py
    ├── 3_1629.py
    ├── 3_10830.py
    ├── 4_3080.py
    ├── 4_5670.py
    ├── 5_2243.py
    ├── 5_3653.py
    └── 5_17408.py

```

풀지 않은 문제도 모두 `convert_for_submission.py` 를 통해 파일을 생성한 뒤, `submission` 폴더 안에 포함해 주세요.

채점 기준

채점은 `submission` 폴더 내 파일을 기준으로 진행됩니다.

아래 조건을 모두 만족해야 정답으로 인정됩니다.

- ☐ 허용된 영역 밖의 코드 수정이 없어야 합니다.
- ☐ 코드가 `mypy` 테스트를 통과해야 합니다.
- ☐ 제공된 input에 대해 정답 output과 일치해야 합니다.

하나라도 충족하지 못할 경우 해당 문제는 오답 처리됩니다.

통과 기준

총 10문제 중 통과한 문제 수를 기준으로 평가합니다.

- **5개 이상:** 통과
- **4개:** 미흡
- **3개 이하:** 실패
- **8개 이상:** 멋져요!!